Práctica 1. Creación de redes con scripts

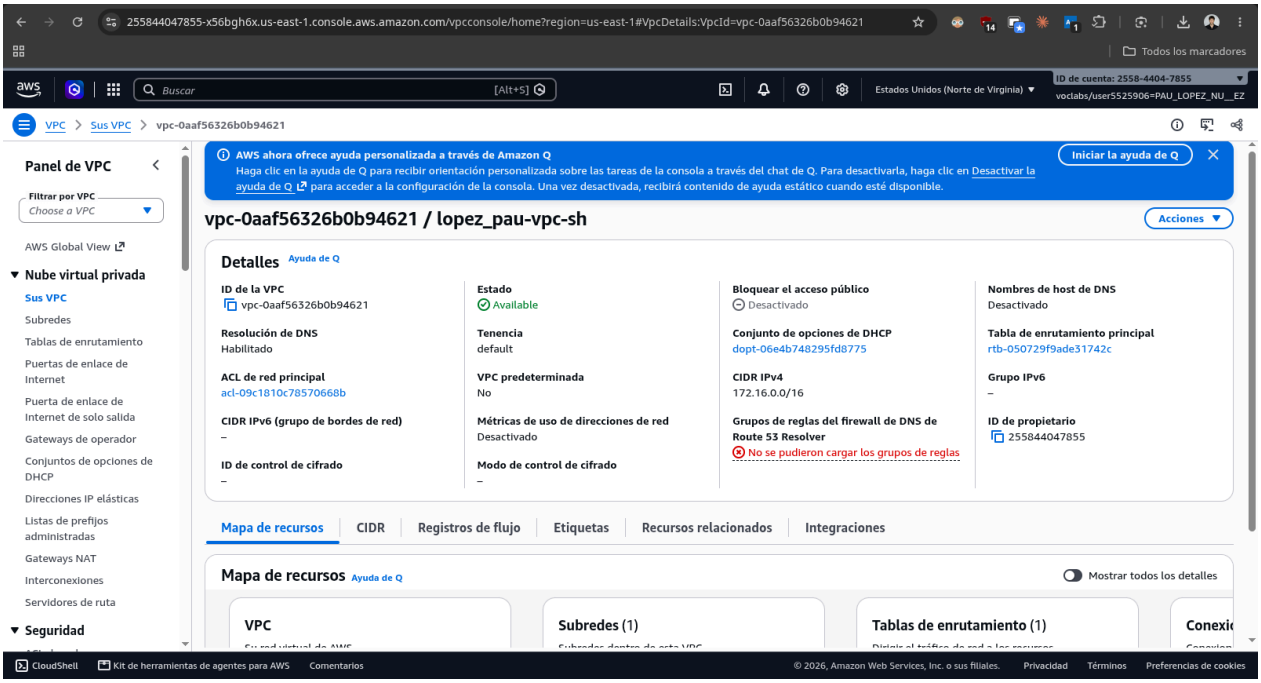
Pau López Núñez. Administración de redes en la nube. 30/10/2026

Cada script crea una VPC (172.16.0.0/16), una subred (172.16.0.0/20) y un EC2 en la región us-east-1.

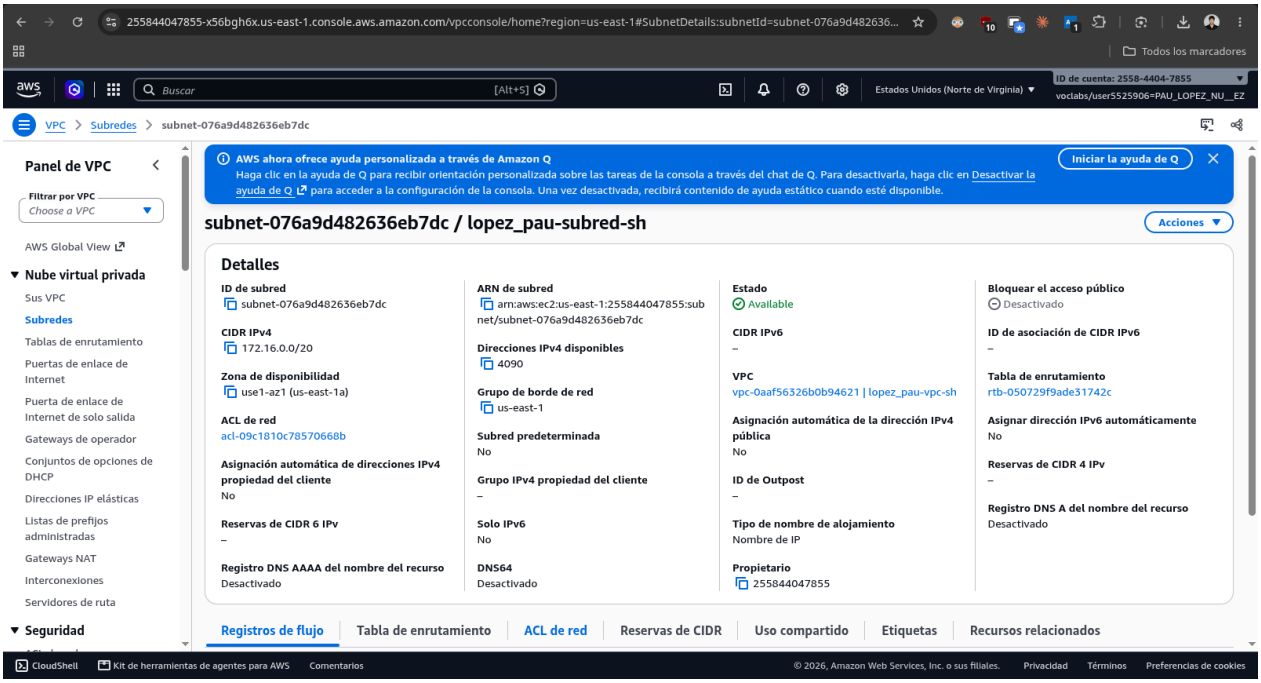
Script con la CLI: crear_vpc.sh

```
$ aws sts get-caller-identity --query Arn --output text
arn:aws:sts::255844047855:assumed-role/voclabs/user5525906=PAU_LOPEZ_NU__EZ
$ ./crear_vpc.sh
VPC creada: vpc-0aaf56326b0b94621
Subred creada: subnet-076a9d482636eb7dc
Instancia creada: i-0ecf67d847ee08154
-----------------------------------------
|           DescribeInstances           |
+---------+---------------+-------------+
| Estado  |  IP_privada   | IP_publica  |
+---------+---------------+-------------+
|  running|  172.16.6.99  |  None       |
+---------+---------------+-------------+
```

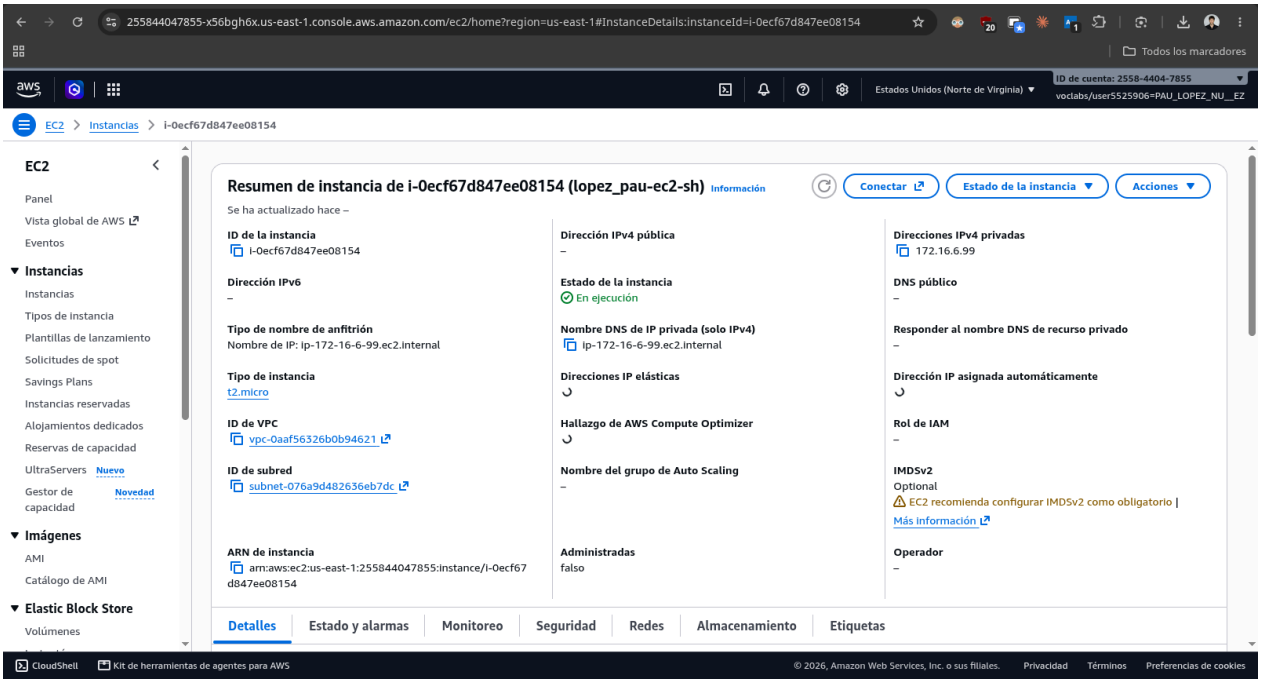
VPC creada en el panel de control.



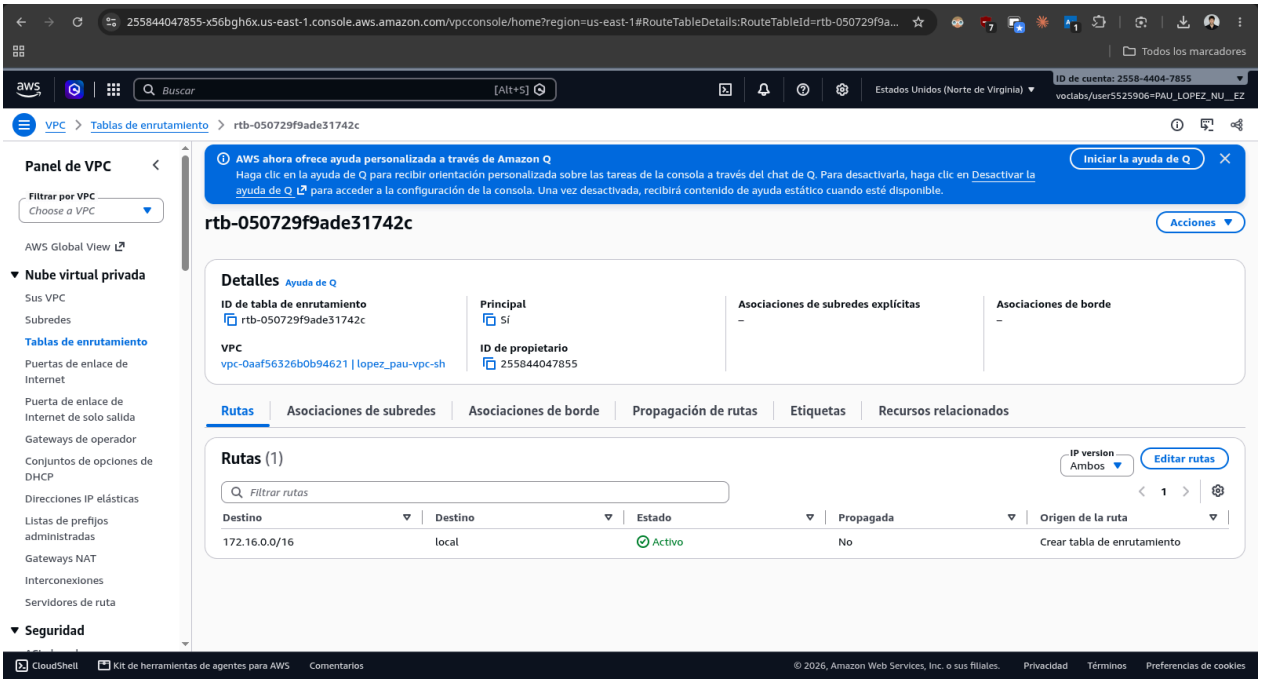
Subred dentro de la VPC.



EC2 en la subred, con IP privada 172.16.6.99 y sin IP pública.



La tabla de rutas solo tiene la ruta local.

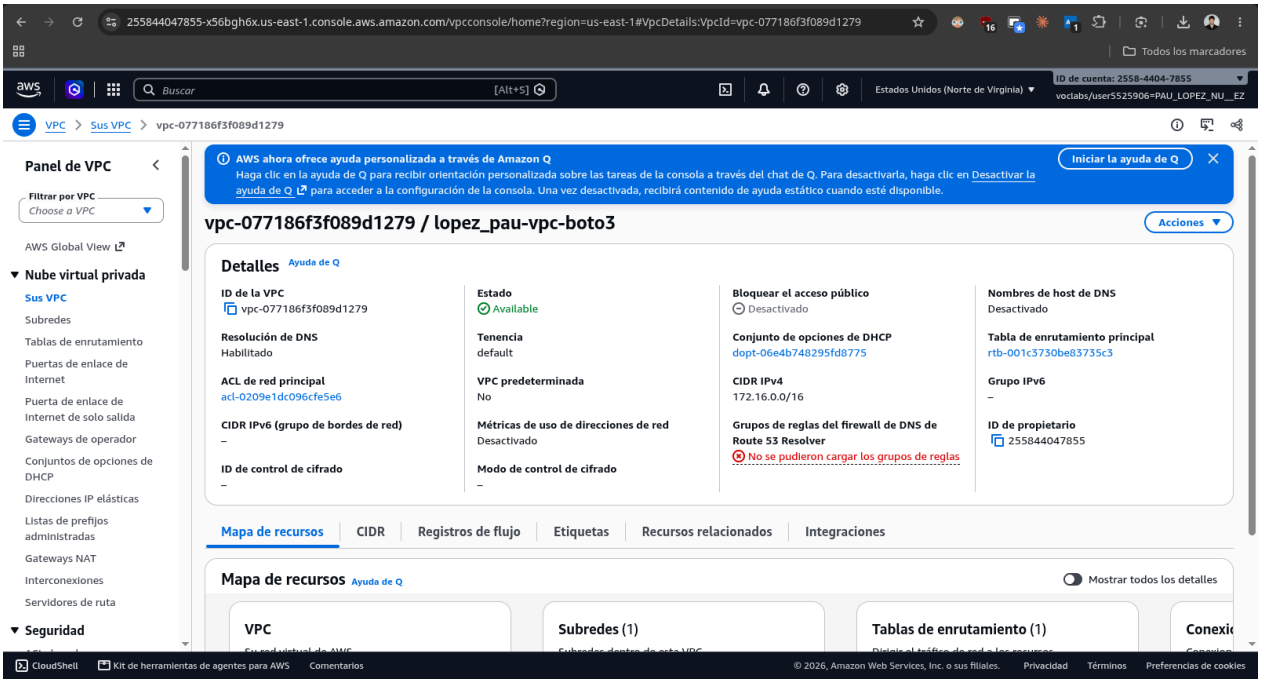


Script con Boto3: crear_vpc.py

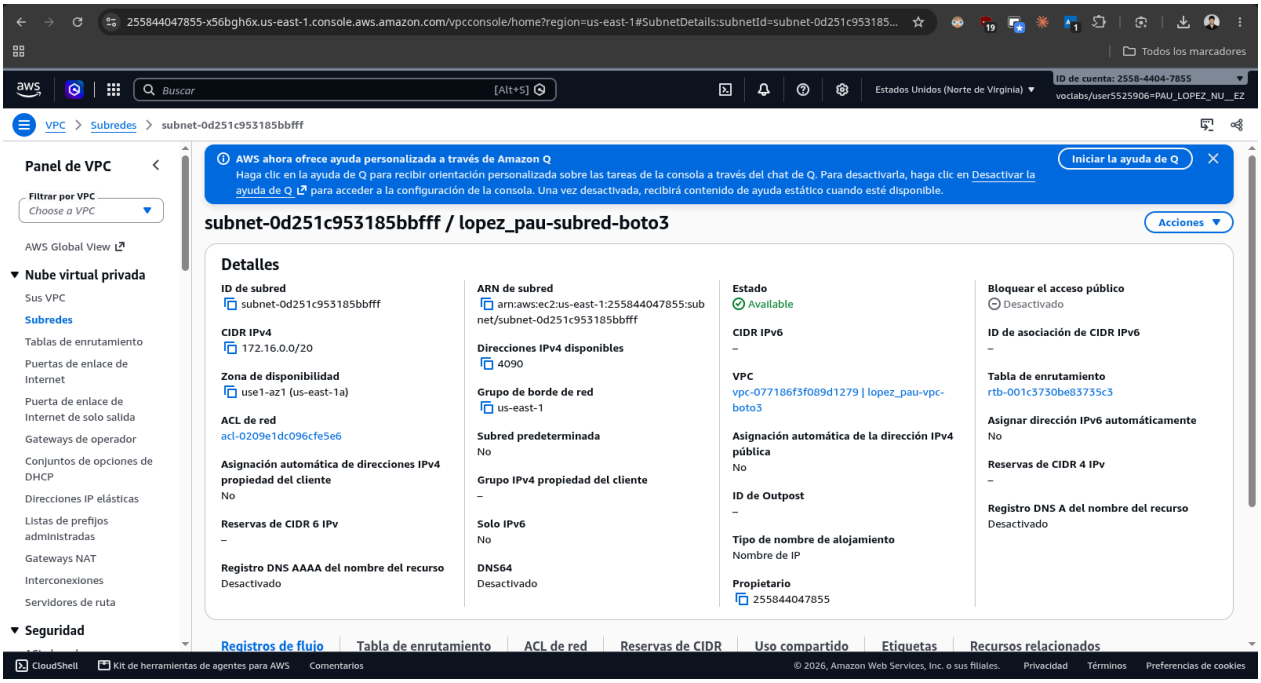
El lab solo permite 5 VPC, así que se borró la VPC del script anterior antes de ejecutar este.

```
$ python3 crear_vpc.py
VPC creada: vpc-077186f3f089d1279
Subred creada: subnet-0d251c953185bbfff
Instancia creada: i-03c74dd93eb22772f
Estado: running
IP privada: 172.16.12.247
IP pública: no tiene
```

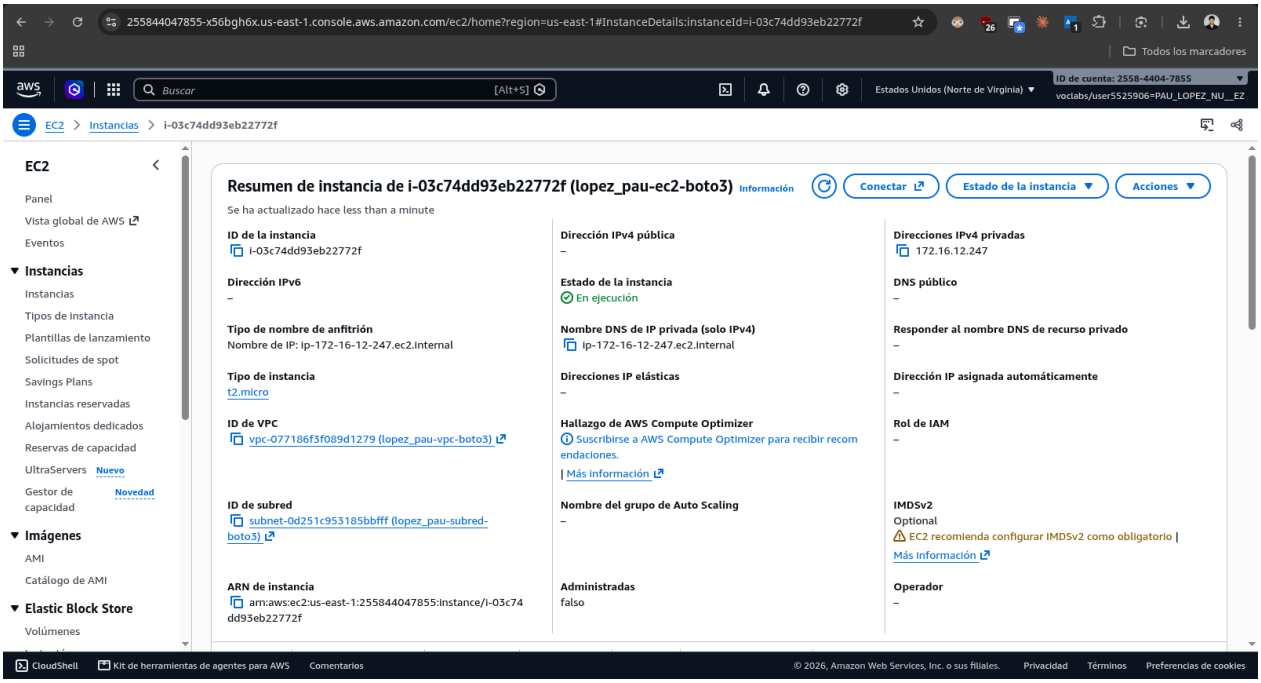
VPC creada en el panel de control.



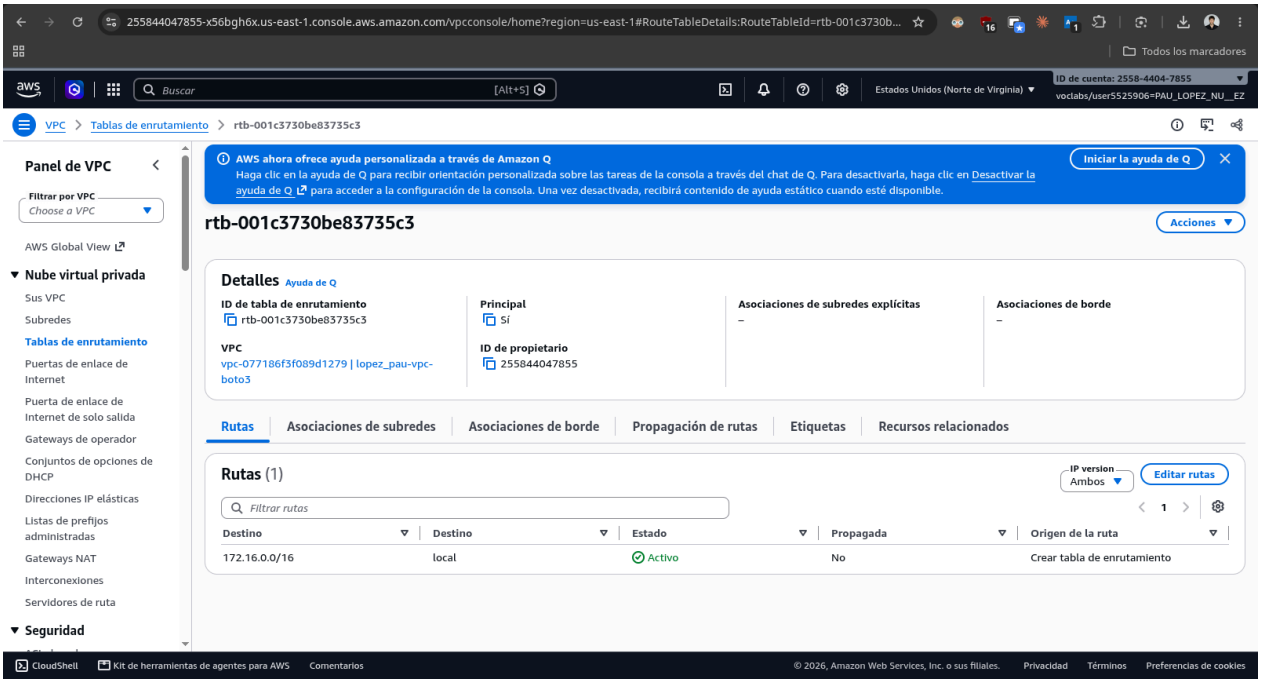
Subred dentro de la VPC.



EC2 en la subred, con IP privada 172.16.12.247 y sin IP pública.



La tabla de rutas solo tiene la ruta local.



Preguntas

Direcciones IP. Las IP privadas (172.16.6.99 y 172.16.12.247) coinciden en el panel y en la terminal, y están dentro de la subred 172.16.0.0/20. Ninguna instancia tiene IP pública.

¿Podemos tener conexión desde el exterior con el EC2? No. La VPC no tiene Internet Gateway y su tabla de rutas solo tiene la ruta local, y además la instancia no tiene IP pública. Por eso tampoco se puede

hacer la conexión SSH con estos scripts. Para tener conexión habría que añadir un Internet Gateway, una ruta 0.0.0.0/0 hacia él, una IP pública y permitir el puerto 22 en el grupo de seguridad.

Scripts

crear_vpc.sh

```
#!/bin/bash
# Práctica 1 - Crea una VPC, una subred y un EC2 dentro con la CLI
# Pau López Núñez

set -e

AMI_ID=ami-033ea4ff4a9c726dc    # Ubuntu Server 22.04 (Canonical)

# 1. Crear la VPC
VPC_ID=$(aws ec2 create-vpc \
    --cidr-block 172.16.0.0/16 \
    --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=lopez_pau-vpc-sh}]' \
    --query 'Vpc.VpcId' \
    --output text)
echo "VPC creada: $VPC_ID"

# 2. Crear la subred dentro de la VPC
SUB_ID=$(aws ec2 create-subnet \
    --vpc-id "$VPC_ID" \
    --cidr-block 172.16.0.0/20 \
    --availability-zone us-east-1a \
    --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=lopez_pau-subred-sh}]' \
    --query 'Subnet.SubnetId' \
    --output text)
echo "Subred creada: $SUB_ID"

# 3. Lanzar el EC2 en la subred
INST_ID=$(aws ec2 run-instances \
    --image-id "$AMI_ID" \
    --instance-type t2.micro \
    --key-name vockey \
    --subnet-id "$SUB_ID" \
    --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=lopez_pau-ec2-sh}]' \
    --query 'Instances[0].InstanceId' \
    --output text)
echo "Instancia creada: $INST_ID"

aws ec2 wait instance-running --instance-ids "$INST_ID"
aws ec2 describe-instances --instance-ids "$INST_ID" \
    --query 'Reservations[0].Instances[0].
{Estado:State.Name,IP_privada:PrivateIpAddress,IP_publica:PublicIpAddress}' \
    --output table
```

crear_vpc.py

```
#!/usr/bin/env python3
# Práctica 1 - Crea una VPC, una subred y un EC2 dentro con Boto3
# Pau López Núñez

import sys

import boto3
from botocore.exceptions import ClientError

REGION = "us-east-1"
ZONA = "us-east-1a"
AMI_ID = "ami-033ea4ff4a9c726dc"  # Ubuntu Server 22.04 (Canonical)

ec2 = boto3.client("ec2", region_name=REGION)
```

```
def nombre(tipo, valor):
    return [{"ResourceType": tipo, "Tags": [{"Key": "Name", "Value": valor}]}]


def crear_vpc():
    vpc_id = ec2.create_vpc(
        CidrBlock="172.16.0.0/16",
        TagSpecifications=nombre("vpc", "lopez_pau-vpc-boto3"),
    )["Vpc"]["VpcId"]
    ec2.get_waiter("vpc_available").wait(VpcIds=[vpc_id])
    print(f"VPC creada: {vpc_id}")
    return vpc_id


def crear_subred(vpc_id):
    subred_id = ec2.create_subnet(
        VpcId=vpc_id,
        CidrBlock="172.16.0.0/20",
        AvailabilityZone=ZONA,
        TagSpecifications=nombre("subnet", "lopez_pau-subred-boto3"),
    )["Subnet"]["SubnetId"]
    print(f"Subred creada: {subred_id}")
    return subred_id


def crear_ec2(subred_id):
    inst_id = ec2.run_instances(
        ImageId=AMI_ID,
        InstanceType="t2.micro",
        KeyName="vockey",
        SubnetId=subred_id,
        MinCount=1,
        MaxCount=1,
        TagSpecifications=nombre("instance", "lopez_pau-ec2-boto3"),
    )["Instances"][0]["InstanceId"]
    print(f"Instancia creada: {inst_id}")

    ec2.get_waiter("instance_running").wait(InstanceIds=[inst_id])
    inst = ec2.describe_instances(InstanceIds=[inst_id])["Reservations"][0]["Instances"][0]
    print(f"Estado: {inst['State']['Name']}")
    print(f"IP privada: {inst['PrivateIpAddress']}")
    print(f"IP pública: {inst.get('PublicIpAddress', 'no tiene')}")


def main():
    try:
        vpc_id = crear_vpc()
        subred_id = crear_subred(vpc_id)
        crear_ec2(subred_id)
        return 0
    except ClientError as e:
        print(f"Error de AWS: {e}", file=sys.stderr)
        return 1


if __name__ == "__main__":
    sys.exit(main())
```